

Revenue Performance & Integrity Analysis

Business Problem Statement

An e-commerce company needs to ensure the accuracy, completeness, and reliability of its revenue data for financial reporting and business decision-making. However, inconsistencies can arise between order-level transaction data and actual customer payments.

Specifically, discrepancies exist between:

- Expected revenue calculated from order items (price + freight value)
- Actual payments received from customers (including installment-based transactions)

Since a single order may contain multiple items and multiple payment transactions, validating revenue accuracy becomes complex.

The core business problem is:

Do customer payments accurately reflect the true value of each order?

Validation Logic:

- Expected Value = $\text{SUM}(\text{price} + \text{freight_value})$
- Actual Value = $\text{SUM}(\text{payment_value})$

Why This Problem is Important

Failure to reconcile revenue accurately can lead to:

- Revenue misreporting in financial statements
- Financial losses due to underpayments
- Customer dissatisfaction in cases of overcharging
- Inefficient decision-making due to unreliable data

Additionally, without proper reconciliation, it becomes difficult to identify operational inefficiencies across regions, order statuses, and customer segments.

Objective

- Validate revenue accuracy across multiple datasets
- Identify mismatches between expected and actual revenue
- Detect missing, duplicate, or inconsistent transactions
- Perform root cause analysis of discrepancies
- Provide actionable insights to improve financial and operational efficiency

Deliverables

1. Data Preparation & Exploration (Python)

- Loaded and explored datasets
- Validated schema, data types, and relationships
- Handled missing values appropriately
- Identified initial anomalies and patterns

2. Data Analysis & Reconciliation (SQL / Python)

- Calculated order-level expected revenue
- Aggregated payment values per order
- Performed reconciliation using order_id
- Classified results into:
 - Match
 - Underpaid
 - Overpaid

3. Business Analysis (SQL)

- Measured mismatch percentage
- Evaluated revenue impact (underpaid vs overpaid)
- Identified high-value mismatched orders
- Analyzed discrepancies by state and order status

4. Visualization (Power BI)

Developed an interactive dashboard featuring:

- KPI monitoring (Revenue, Mismatch %, Revenue Gap)
- Mismatch analysis by state, order status, and time
- Customer contribution analysis (Top 10 customers < 1%)
- Risk-based conditional formatting and alerts
- Action-oriented insights and recommendations

5. GitHub Repository

Python notebooks (EDA & analysis)

SQL queries and stored procedures

Power BI dashboard

Clean and structured README documentation